Lecture 5: Bitwise Mechanics, Control Flow, and Foundational Algorithms

1. Bitwise Operators & Mechanics

Bitwise operations execute at the lowest memory level, manipulating the raw binary representation of data. These are highly efficient and heavily utilized in competitive programming for constant-time $O(1)$ math and state manipulations.

- **AND (`&`):** Yields 1 if and only if both corresponding bits are 1. Useful for masking (e.g., `n & 1` extracts the Least Significant Bit).
- **OR (`|`):** Yields 1 if at least one of the corresponding bits is 1. Useful for setting specific bits.
- **XOR (`^`):** Yields 1 if the corresponding bits are strictly different. A critical property: $x \oplus x = 0$ and $x \oplus 0 = x$.
- **NOT (`~`):** A unary operator that flips all bits (1s complement). Note: In a 32-bit signed integer system, taking the NOT of a positive number will flip the sign bit, returning a negative number represented in 2's complement form.

Shift Operators:

- **Left Shift (`<<`):** Shifts the bit pattern to the left by a specified number of positions, padding the right side with `0`s. Mathematically, $a \ll b$ is equivalent to $a \times 2^b$.
- **Right Shift (`>>`):** Shifts the bit pattern to the right. Mathematically, $a \gg b$ is equivalent to $\lfloor a / 2^b \rfloor$.
  - *Padding Rule (Positive Numbers):* Always padded with `0`s on the left.
  - *Padding Rule (Negative Numbers):* Compiler-dependent, but standard behavior usually pads with `1`s to preserve the negative sign bit.

Operator	Symbol	Mathematical Equivalent	Typical Use Case
Bitwise AND	`&`	N/A	Masking, checking odd/even (`n & 1`)
Left Shift	`<<`	$N \times 2^K$	Fast multiplication by powers of 2
Right Shift	`>>`	$\lfloor N / 2^K \rfloor$	Fast division by powers of 2
XOR	`^`	N/A	Toggling bits, finding unique elements

2. Loop Control Flow & Variable Scope

Mastering how the execution pointer moves through loops prevents infinite cycles and memory leaks.

- **The `for` Loop:** Consists of three optional components: `for(initialization; condition; update)`. You can declare multiple variables (e.g., `int a = 0, b = 1`) or omit the condition completely (creating an infinite loop that must be broken internally).
- **The `break` Keyword:** Immediately terminates the closest enclosing loop, handing control to the very next line of code outside the loop block.
- **The `continue` Keyword:** Immediately halts the current iteration, skipping the rest of the code block. It jumps directly to the **update** expression (in a `for` loop) or condition evaluation, proceeding with the next iteration.
- **Block Scope (`{}`):** A variable's lifecycle is strictly bound to the curly braces in which it is declared. If you declare `int i = 0` inside an `if` block, `i` is immediately destroyed in memory once that block finishes.

Problem-Solving Deconstruction

Problem 1: Print Fibonacci Sequence

1. Problem Statement & Constraints: Generate and print the first N terms of the Fibonacci series, where $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$.

2. Core Intuition: We only need to track the two most recent numbers to calculate the next one. We iteratively calculate the sum, print it, and then explicitly shift our two state variables forward.

3. Algorithmic Steps: 1. Initialize `a = 0` and `b = 1`. 2. Print `a` and `b` as the base foundations. 3. Start a `for` loop from $i = 1$ up to N. 4. Calculate `nextNumber = a + b`. 5. Print `nextNumber`. 6. Shift states: `a = b` and `b = nextNumber`.

4. Dry Run:

Input: $N = 5$

1. Print `0 1`.
2. $i = 1$: `next = 0 + 1 = 1`. Print `1`. Shift: `a=1, b=1`.
3. $i = 2$: `next = 1 + 1 = 2`. Print `2`. Shift: `a=1, b=2`.
4. $i = 3$: `next = 1 + 2 = 3`. Print `3`. Shift: `a=2, b=3`.
5. $i = 4$: `next = 2 + 3 = 5`. Print `5`. Shift: `a=3, b=5`.
6. $i = 5$: `next = 3 + 5 = 8`. Print `8`. Loop ends.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	Iterates exactly N times to compute the sequence.
Space Complexity	$O(1)$	Constant memory used for variables `a`, `b`, and `nextNumber`.

C++

```
#include<iostream>
using namespace std;

int main() {
    int n = 10;
    int a = 0;
    int b = 1;
    cout << a << " " << b << " "; // Print base states

    for(int i = 1; i <= n; i++) {
        int nextNumber = a + b;
        cout << nextNumber << " ";

        // State shift
        a = b;
        b = nextNumber;
    }
    return 0;
}
```

Problem 2: LeetCode 1281 - Subtract Product and Sum of Digits

1. Problem Statement & Constraints: Given an integer N, return the difference between the product of its digits and the sum of its digits.

2. Core Intuition: This is a standard base-10 digit extraction algorithm. Any integer modulo 10 (`n % 10`) isolates its least significant digit (rightmost digit). Dividing the integer by 10 (`n / 10`) permanently drops that digit. We loop this mechanism until the integer decays to 0.

3. Algorithmic Steps:

1. Initialize `prod = 1` and `sum = 0`.
2. Create a `while` loop that continues as long as `n != 0`.
3. Extract the last digit: `int digit = n % 10`.
4. Update product: `prod = prod * digit`.
5. Update sum: `sum = sum + digit`.
6. Reduce the number: `n = n / 10`.
7. Return `prod - sum`.

4. Dry Run:

Input: $N = 234$

1. `digit = 4`. `prod = 4`, `sum = 4`. `N = 23`.
2. `digit = 3`. `prod = 12`, `sum = 7`. `N = 2`.
3. `digit = 2`. `prod = 24`, `sum = 9`. `N = 0`. Loop breaks.
4. Output: $24 - 9 = 15$.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\log_{10} N)$	The loop executes once per digit. The number of digits in N is roughly $\log_{10}(N)$.
Space Complexity	$O(1)$	Only scalar variables `sum`, `prod`, and `digit` are tracked.

C++

```
class Solution {
public:
    int subtractProductAndSum(int n) {
        int prod = 1;
        int sum = 0;

        while(n != 0) {
            int digit = n % 10;          // Isolate LSB in base 10
            prod = prod * digit;         // Accumulate product
            sum = sum + digit;           // Accumulate sum
            n = n / 10;                  // Discard LSB
        }

        int answer = prod - sum;
        return answer;
    }
};
```

Problem 3: LeetCode 191 - Number of 1 Bits (Hamming Weight)

1. Problem Statement & Constraints: Given an unsigned integer N, count and return the total number of set bits (bits evaluated to `1`).
2. Core Intuition: Instead of base-10 extraction, we use base-2 (bitwise) extraction. The expression `n & 1` acts as a bitmask that evaluates to `1` if the Least Significant Bit (LSB) is `1`, and `0` otherwise. After checking, we right-shift the entire bit sequence (`n >> 1`) to push the next bit into the LSB position.
3. Algorithmic Steps: 1. Initialize a `count = 0`. 2. Start a `while` loop that runs as long as `n != 0`. 3. Check the LSB: If `n & 1` is true,

increment `count`. 4. Right shift N by 1 bit: `n = n >> 1`. 5. Return the `count` when N becomes 0 (all 1s have been processed and shifted out).

4. Dry Run:

Input: $N = 11$ (Binary: `1011`)

1. LSB of `1011` is `1`. `count = 1`. Shift right: `n = 101`.
2. LSB of `101` is `1`. `count = 2`. Shift right: `n = 10`.
3. LSB of `10` is `0`. `count = 2`. Shift right: `n = 1`.
4. LSB of `1` is `1`. `count = 3`. Shift right: `n = 0`. Loop breaks.
5. Output: `3`.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(1)$	In a standard 32-bit integer system, the loop will run at most 32 times regardless of input size. Thus, it operates in strict constant time.
Space Complexity	$O(1)$	Constant memory allocation for the `count` variable.

C++

```
class Solution {
public:
    int hammingWeight(uint32_t n) {
        int count = 0;

        while(n != 0) {
            // Mask the last bit to check if it's 1
            if(n & 1) {
                count++;
            }
            // Logical right shift by 1 to bring the next bit to the LSB
position
            n = n >> 1;
        }
        return count;
    }
};
```